

	no se modifica.								
[ESC]M	Suprimir línea. La línea en la que está el cursor se pierde; todas las siguientes se desplazan hacia arriba; se borra la última línea; la posición del cursor no se modifica.								
[ESC]N	Suprimir carácter. Todos los caracteres que están a la derecha del cursor, en su misma línea, se desplazan una posición hacia la izquierda. El carácter de la última columna queda en blanco.								
[ESC]Y<I> <c>	Lleva el cursor a la posición especificada. Si esa posición sobrepasa algún borde de la pantalla, el cursor se queda en el borde correspondiente. <I>=línea+ &20; <c>=columna+ &20. El extremo superior izquierdo es línea 0, columna 0.								
[ESC]b<pc>	Establece color de pluma. Afecta a todos los caracteres presentes en la pantalla. El parámetro de color, <pc>, se enmascara con &3F y luego se lo interpreta como tres números de 2 bits que especifican las intensidades de los tres colores primarios: los bits 0 y 1 para el azul, los bits 2 y 3 para el rojo y los bits 4 y 5 para el verde. El 6128 proporciona tres niveles de intensidad, que se especifican de la siguiente forma: <table><tr><td>CPC6128</td><td>Intensidad nula</td><td>Intensidad media</td><td>Intensidad máxima</td></tr><tr><td>Bits para cada color</td><td>00 (binario)</td><td>01 ó 10 (binario)</td><td>11 (binario)</td></tr></table>	CPC6128	Intensidad nula	Intensidad media	Intensidad máxima	Bits para cada color	00 (binario)	01 ó 10 (binario)	11 (binario)
CPC6128	Intensidad nula	Intensidad media	Intensidad máxima						
Bits para cada color	00 (binario)	01 ó 10 (binario)	11 (binario)						
[ESC]c<pc>	Establece color de fondo. Afecta tanto al fondo como al borde de la pantalla. Los colores se especifican como en la secuencia anterior.								
[ESC]d	Borrar desde el principio de la página (hasta la posición del cursor inclusive). No se modifica la posición del cursor.								
[ESC]e	Activa el cursor. Para evitar el molesto parpadeo que de otra forma se produciría, el cursor está inhibido durante la escritura normal del texto y sólo se lo activa una décima de segundo después de escribir el último carácter.								
[ESC]f	Inhibe el cursor.								
[ESC]j	Almacenar la posición del cursor.								
[ESC]k	Restablecer la posición del cursor almacenada por [ESC]j.								
[ESC]l	Borrar línea. No se modifica la posición del cursor.								
[ESC]o	Borrar desde el principio de la línea (hasta la posición del cursor inclusive). No se modifica la posición del cursor.								
[ESC]p	Activar el modo de imagen invertida ('video inverso'). Los caracteres visibles se escriben con los colores de fondo y pluma invertidos.								
[ESC]q	Desactivar el modo de imagen invertida.								
[ESC]r	Activar el modo de subrayado. (No disponible en el CPC6128.)								
[ESC]u	Desactivar el modo de subrayado. (No disponible en el CPC6128.)								

[ESC]v	Activa la continuidad de las líneas. Cuando los caracteres enviados a la pantalla sobrepasan la última columna de una línea, siguen apareciendo al principio de la línea siguiente.
[ESC]w	Desactiva la continuidad de las líneas. Cuando los caracteres enviados a la pantalla sobrepasan la última columna de una línea, se los ignora.
[ESC]x	Activa el modo 24x80. Algunos programas requieren la pantalla estándar de 24 filas por 80 columnas. Esta secuencia selecciona esa pantalla, independientemente de cuál sea el tamaño total de la pantalla (que depende de la máquina), del país y de si la línea de estado está activada o desactivada. Se borra la pantalla.
[ESC]y	Desactiva el modo 24x80. Se borra la pantalla.

Parte 17: Juego de caracteres de CP / M Plus

En la parte 10 de este capítulo hemos descrito una tabla de traducción para impresoras. El fin de esta tabla es convertir algunos caracteres del juego de BASIC y CP/M 2.2 a una forma tal que puedan ser escritos por la impresora cuando en ella se ha realizado cierta selección de idioma. Esta función está limitada por el hecho de que pocos de los caracteres internacionales de las impresoras aparecen en el juego de caracteres de BASIC.

Aunque este método sigue funcionando en CP/M Plus, se lo ha ampliado hasta permitir una correspondencia casi absoluta entre los caracteres visibles en la pantalla y los de la impresora. (El carácter visible que falta es el signo monetario sueco, que debería figurar en lugar de '\$'.) La distribución de caracteres por idiomas es la siguiente:

	23	24	40	5B	5C	5D	5E	60	7B	7C	7D	7E
EE. UU.	£	¤	¥	₣	₧	₡	₢	₣	₧	₡	₢	₣
Francia	£	¤	¥	₣	₧	₡	₢	₣	₧	₡	₢	₣
Alemania	£	¤	¥	₣	₧	₡	₢	₣	₧	₡	₢	₣
Reino Unido	£	¤	¥	₣	₧	₡	₢	₣	₧	₡	₢	₣
Dinamarca	£	¤	¥	₣	₧	₡	₢	₣	₧	₡	₢	₣
Suecia	£	¤	¥	₣	₧	₡	₢	₣	₧	₡	₢	₣
Italia	£	¤	¥	₣	₧	₡	₢	₣	₧	₡	₢	₣
España	£	¤	¥	₣	₧	₡	₢	₣	₧	₡	₢	₣

Juego internacional de caracteres de CP/M Plus

Para adaptar la máquina a un idioma determinado se debe hacer lo siguiente:

1. Seleccionar el idioma en la impresora (generalmente mediante microconmutadores basculantes, aunque en algunas impresoras la selección se puede realizar por programa enviándole códigos de control).
2. Seleccionar el juego de caracteres deseado, bien mediante la orden transitoria

LANGUAGE <n>

bien enviando la secuencia

[ESC] 2<n>

al emulador de consola.

En la práctica, esta inicialización se suele programar en el procedimiento PROFILE.SUB mediante las funciones LANGUAGE y SETKEYS. El CP/M Plus se suministra con el juego de caracteres número 0 (EE.UU.) preseleccionado, ya que en la tecla del '3' está grabado el signo '#'.

Juegos de caracteres de 7 bits

Por útil que sea esta posibilidad de seleccionar juegos de caracteres, debemos resaltar que los caracteres estándar (EE.UU.) reemplazados por caracteres de otros idiomas dejan de ser visualizables. Esto es inevitable cuando se manejan juegos de caracteres de 7 bits. Prácticamente todos los programas disponibles para CP/M Plus (incluidos casi todos los programas de ayuda y los procesadores de texto) trabajan con un juego de caracteres de 7 bits. En el Reino Unido se acepta que el signo '#' desaparezca para ser sustituido por el '£', no sólo en proceso de textos, donde es deseable, sino también en los listados de programas, v.g. LIST £8, donde no lo es. Con el tiempo el usuario llega a habituarse a hacer mentalmente la transformación de un signo a otro.

Pero en otros idiomas los cambios son más perturbadores. Los signos nuevos reemplazan caracteres tan necesarios como la barra vertical y los corchetes. A pesar de la mejora de la legibilidad de los textos gracias a los acentos, en las situaciones en las que el programa requiere la barra y los corchetes (por ejemplo, DIR [ALL] la legibilidad queda notablemente reducida. Lo mismo se puede decir de la facilidad de escritura, si se han cambiado algunas teclas para adaptarlas al nuevo juego de caracteres.

Recuerde que los programas trabajan con códigos ASCII y son totalmente indiferentes a la forma que puedan tener los caracteres en la pantalla. Lo que ocurre con los juegos de caracteres de 7 bits es, sencillamente, que no hay códigos suficientes para cubrir todas las necesidades.

Juegos de caracteres de 8 bits

El juego de caracteres de BASIC tiene 256 símbolos diferentes; los códigos del 128 al 255 (&80 a &FF) representan diversos símbolos gráficos de aplicación en programas de juegos e informática doméstica. En CP/M Plus también hay 256 caracteres, pero los 128 últimos son distintos de los de BASIC; están diseñados para aplicaciones comerciales y para la generación de juegos específicos por idiomas:

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	▢	▣	▤	▥	▦	▧	▨	▩	▪	▫	▬	▭	▮	▯	▰	▱
1	▲	△	▴	▵	▶	▷	▸	▹	►	▻	▼	▽	▾	▿	▿	▿
2	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿
3	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
4	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p
5	q	r	s	t	u	v	w	x	y	z	[	\	]	^	_	`
6	~	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	[	\	]	^	_

Caracteres del 0 al 127 (&00 a &7F)

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
8	▢	▣	▤	▥	▦	▧	▨	▩	▪	▫	▬	▭	▮	▯	▰	▱
9	▲	△	▴	▵	▶	▷	▸	▹	►	▻	▼	▽	▾	▿	▿	▿
A	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿
B	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿
C	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿
D	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿
E	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿
F	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿	▿

Caracteres del 128 al 255 (&80 a &FF)

Juego de caracteres estándar (EE.UU.) de CP/M Plus

Los programas capaces de trabajar con juegos de caracteres de 8 bits disponen de este juego completo y podrán, no sólo dibujar recuadros, sino también acceder a los caracteres especiales de todos los idiomas, sin encontrar los problemas que describíamos en la sección anterior.

En la actualidad hay muy pocos programas preparados para trabajar con juegos de caracteres de 8 bits. Los caracteres de los márgenes de 0 a 31 y de 128 a 255 tendrán el aspecto que se muestra en la figura anterior solamente en la pantalla. En la impresora serán distintos, pues los fabricantes de impresoras tienen sus propias ideas sobre el diseño de juegos de caracteres.

Capítulo 8 - Más sobre el gestor de bancos de memoria

Utilización por BASIC del segundo banco de 64K de RAM.

Temas tratados:

- * Almacenamiento de imágenes de pantalla
- * Pseudoficheros

El mapa de memoria de BASIC 1.1 que vimos en el capítulo anterior, parte 14, muestra que BASIC no utiliza 64 de los 128K de RAM. BASIC y el firmware residen en una ROM; si a ésta se añade la ROM del sistema de disco, resulta que la memoria utilizable desde BASIC es 112K en total (64K de RAM y 48K de ROM).

Cada sección de 16K es un 'bloque'; un grupo de 4 bloques (64K) es lo que se denomina 'banco'. La técnica de selección de bloques se llama, pues, 'conmutación de bancos'.

El microprocesador Z80 no puede direccionar, en un momento dado, más de 64K. Por esa razón el sistema operativo está preparado para utilizar la ROM del firmware en lugar del bloque 0 de RAM, o las ROM de BASIC o del sistema de disco en lugar del bloque 3 de RAM. Esta conmutación se realiza automáticamente cada vez que se deben ejecutar rutinas de BASIC o del firmware.

Pues bien, la selección de bancos de RAM no es más que una generalización de este método. La conmutación la realiza un programa escrito en ensamblador.

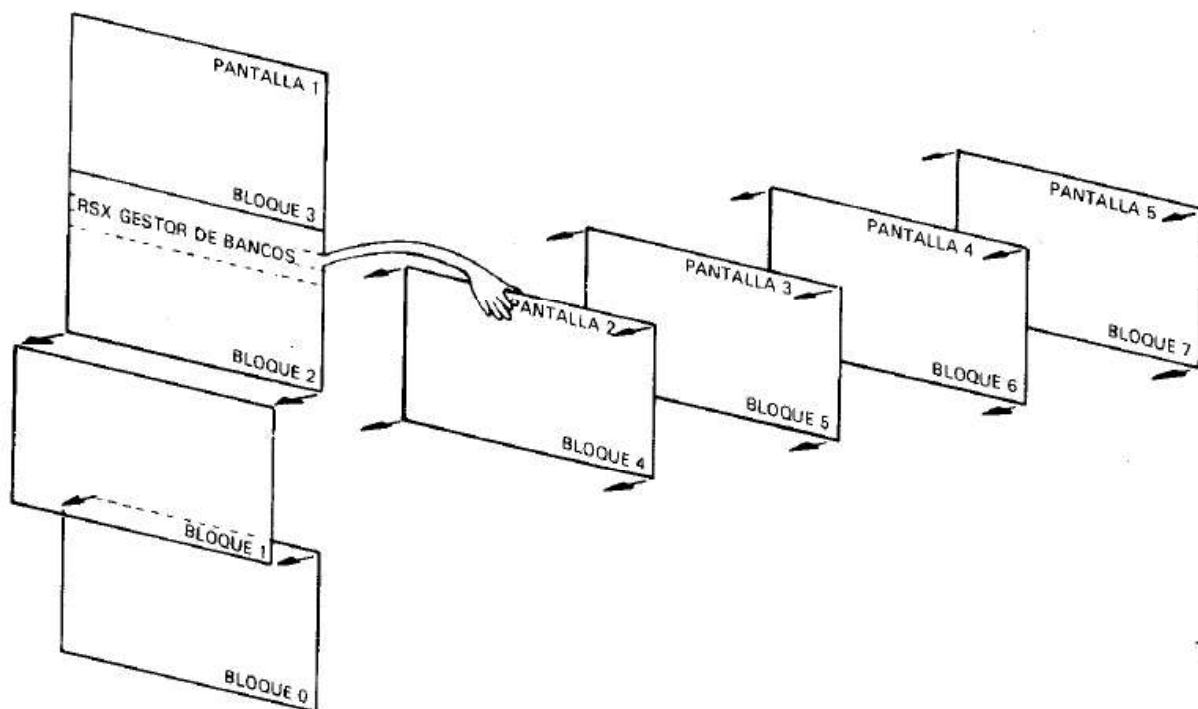
En la cara 1 del juego de discos del sistema se incluye el programa BANKMAN.BAS. Este programa se ejecuta desde BASIC; su objeto es instalar la RSX que proporciona las órdenes externas de gestión de bancos.

Una aplicación del segundo banco de 64K es el almacenamiento temporal de imágenes de pantalla; por ejemplo, en un programa 'diseñador de pantallas' que almacene varias pantallas diferentes, o un videojuego que tenga preparadas varias pantallas para mostrarlas inmediatamente cuando las necesite.

Otra aplicación del segundo banco de 64K es la ampliación del espacio dedicado a las variables; este espacio adicional se puede considerar como ampliación del espacio reservado a las matrices literales o como "disco de RAM" ('Ramdisc').

Parte 1: Almacenamiento de imágenes de pantalla

El gestor de bancos puede 'desconectar' el bloque 1 de RAM y poner en su lugar uno cualquiera de los cuatro bloques de 16K de que consta el segundo banco. La figura siguiente muestra en qué consiste esta conmutación. Observe que los cuatro bloques del segundo banco están todos en el mismo grupo de direcciones, de &4000 a &7FFF. El contenido del bloque 1 (normalmente ocupado por instrucciones de BASIC) no se pierde, sino que es restaurado cuando el gestor de bancos termina su trabajo. Además de las cinco conmutaciones de bancos ilustradas en la figura, hay otras tres posibles, pero sólo se las utiliza en CP/M.



Conmutación de bancos por hardware

El gestor de bancos dispone de dos órdenes externas con las que se pueden transferir pantallas completas de un bloque a otro; estas órdenes conmutan automáticamente los bloques 4 a 7 cuando es necesario y dejan restablecido el bloque 1.

La orden

```
|SCREENSWAP,[<sección de pantalla>,<número de pantalla>,<numero de pantalla>
```

intercambia el contenido de dos bloques, mientras que

```
|SCREENCOPY,[<sección de pantalla>,<numero de pantalla destino>,<numero de  
pantalla fuente>
```

copia el contenido de un bloque en otro.

El parámetro opcional <sección de pantalla> hace que la transferencia de información se realice por tramos de 1/64 de pantalla (256 bytes del total de 16K). Así pues, el parámetro puede tener cualquier valor comprendido entre 0 y 63. Este método de operación es útil cuando se necesita mezclar alguna otra tarea con la de transferencia de pantallas. El tiempo de ejecución de un intercambio de pantallas es de aproximadamente 150/300 segundos (150 unidades de la variable TIME).

Los <números de pantalla> son números cualesquiera del margen de 1 a 5. (La pantalla 1 es la visible.) Las operaciones de copia e intercambio en las que interviene la pantalla 1 son mucho más rápidas que las otras. Tenga cuidado con el efecto de 'desplazamiento vertical por hardware', que ya puede haber experimentado al trabajar con volcados de pantalla. Se debe procurar que todas las imágenes de pantalla se construyan (y visualicen) con la pantalla 1 en la misma posición de hardware. La posición más sencilla es la establecida por una orden MODE.

Ejemplos de transferencias de pantallas

Cargue y ejecute el programa BANKMAN con la orden

```
RUN "BANKMAN"
```

Escriba

```
MODE 1
```

con lo que se borra la pantalla. Ahora escriba

```
' Esta es la pantalla original  
| SCREENCOPY,3,1 'Pantalla original va a la memoria 3  
CLS
```

La pantalla se ha vuelto a borrar. Escriba ahora

```
' Esta es la pantalla intermedia  
| SCREENCOPY,2,1 'Pantalla intermedia va a memoria 2  
| SCREENSWAP,2,3 'Intercambia memorias 2 y 3  
| SCREENCOPY,1,3 'Recupera pantalla intermedia desde memoria 3  
| SCREENSWAP,2,3 'Recupera pantalla original desde memoria 2
```

Al final del capítulo 9 damos un completo programa 'diseñador de pantallas' que utiliza las funciones de transferencia de pantallas proporcionadas por el gestor de bancos.

Parte 2: Ficheros en RAM

Considerado como 'RAMdisc', el segundo banco de 64K de RAM se puede organizar en forma de fichero, con cierto número de registros de longitud fija. La longitud de registro puede ser de entre 0 y 255 bytes, si bien 2 bytes es el mínimo recomendable. Una vez establecida la 'longitud de registro', cada registro es accesible para operaciones de escritura y lectura por su 'número de registro'. Es perfectamente posible escribir en un fichero de RAM con una longitud y luego leer con otra distinta.

Nota. El fichero de RAM sólo puede contener datos; no está prevista su utilización para almacenar instrucciones de programa.

Al igual que en los sistemas de ficheros de acceso aleatorio en disco, aquí existe también el concepto de 'número de registro actual'. Este número se mantiene siempre actualizado, y es útil, por consiguiente, cuando se recorre el fichero avanzando automáticamente de cada registro al siguiente.

La orden

```
|BANKOPEN,<longitud de registro>
```

establece la longitud de registro e inicializa el 'número de registro actual' (a 0), pero no borra memoria.

La orden

```
|BANKWRITE,@<código generado>,<expresión literal>[,<número de registro>]
```

escribe la <expresión literal> en el fichero. El <número de registro> especifica en qué registro

se debe escribir. Si se omite este parámetro, se escribe en el número de registro actual. Finalmente, se actualiza este número (sumándole 1).

Si la <expresión literal> no llena el registro, los caracteres antiguos sobre los que no se ha escrito seguirán estando en el registro. Si la <expresión literal> es demasiado larga, se ignoran los caracteres restantes para no 'corromper' el registro siguiente.

El <código generado> es una variable entera a través de la cual se puede conocer, al concluir la operación, el número de registro en el que se ha escrito (si la escritura se ha realizado con éxito) o un número de error (negativo) si la escritura ha fracasado:

- -1 indica que se ha alcanzado el final del fichero (se ha intentado sobrepasar el final de los 64K)
- -2 indica un error de conmutación de bancos (no debería ocurrir nunca)

Ejemplos:

```
| BANKOPEN,10
| BANKWRITE,@r%,"123 pruebas",0
| BANKWRITE,@r%,w$
```

La orden

```
|BANKREAD,@<código generado>,@<variable literal>[,número de registro]
```

lee un registro y asigna su contenido a la <variable literal>.

El <número de registro> especifica en qué registro se debe leer. Si se omite este parámetro, se lee en el número de registro actual. Finalmente, se actualiza este número (sumándole 1).

Si el contenido del registro no llena completamente la <variable literal>, los caracteres antiguos sobre los que no se ha escrito seguirán estando en la <variable literal>. Si el contenido del registro es más largo que la <variable literal>, se ignoran los caracteres restantes, ya que no es posible aumentar la longitud de la variable mediante una orden externa.

El <código generado> es una variable entera a través de la cual se puede conocer, al concluir la operación, el número de registro en el que se ha leído (si la lectura se ha realizado con éxito) o un número de error (negativo) si la lectura ha fracasado:

- -1 indica que se ha alcanzado el final del fichero (se ha intentado sobrepasar el final de los 64K)
- -2 indica un error de conmutación de bancos (no debería ocurrir nunca)

Ejemplo:

```
|BANKREAD,@r%,i$,0
```

Búsquedas

Se puede recorrer el fichero en busca de un dato determinado. La orden

```
|BANKFIND,@<código generado>,<cadena buscada>[,<numero de registro
  inicial>[,<número de registro final>]]
```

examina los registros especificados. El <número de registro inicial> especifica el registro a

partir del cual debe comenzar la búsqueda. Si se omite este parámetro, se toma por defecto el número de registros actual.

La búsqueda continúa, a saltos de <longitud de registro>, hasta que se encuentra la cadena o hasta que se agotan los 64K.

Si se especifica el <número de registro final>, la búsqueda termina cuando se ha examinado ese registro (a menos que se haya encontrado la cadena antes).

Si la búsqueda concluye con éxito, el 'número de registro actual' apunta al registro donde se ha encontrado la cadena; si no, sigue teniendo el valor anterior.

El <código generado> es una variable entera que contiene, al terminar la operación, información sobre cómo se ha desarrollado ésta. Si ha concluido con éxito, contiene el número de registro en el que encontró la cadena; si no, contiene un número negativo que es un código de error:

- -1 Fin del fichero. El <número de registro inicial> sobrepasa el final de los 64K, o bien es mayor que el <número de registro final>.
- -2 Error de conmutación de bancos (no debería ocurrir nunca).
- -3 No se ha encontrado la cadena buscada.

La <cadena buscada> puede contener símbolos comodín, que en este caso son caracteres número 0, chr\$(0). El número de caracteres que intervienen en las comparaciones es igual a la <longitud de registro> o a la longitud de la <cadena buscada>, el más corto de los dos.

Ejemplos:

```
| BANKFIND,@r%,"123pru",0
| BANKFIND,@r%,f$,100,200
```

Cuidado con los errores

Los errores obvios, tales como poner demasiados parámetros, son detectados e identificados como Bad Command. Sin embargo, el mecanismo de las órdenes externas no permite detectar errores del tipo Type mismatch, por lo que el usuario debe cerciorarse de que está utilizando parámetros de la forma correcta.

El siguiente programa utiliza las órdenes de RAMdisc para examinar una base de datos que contiene permutaciones de letras formadas con palabras de 7 letras. Busca concordancias y permite utilizar símbolos comodín.

Por ejemplo, las permutaciones de la palabra CENTRAL que concuerdan con ?NACE?? (puede omitir los dos últimos símbolos ?) son TNACERL, TNACELR, RNACETL, RNACELT, LNACETR y LNACERT.

Se tarda bastante en crear la base de datos, pero es que 64K es mucha memoria.

```
10 ' REVUELTO DE LETRAS, por ROLAND PERRY
20 ' copyright (c) AMSOFT 1985
30 '
40 ' No olvide RUN "BANKMAN" antes de ejecutar este programa
50 ' *****
60 '
70 MODE 2
```

```

80 DEFINT a-z
90 r%=0:|BANKOPEN,7
100 INPUT"Que palabra de 7 letras quiere revolver";s$
110 IF LEN(s$)<>7 THEN 100
120 PRINT"Por favor, espere un rato."
130 LOCATE 1,5:PRINT"Sigo trabajando:"
140 FOR c1=1 TO 7
150 FOR c2=1 TO 7
160 IF c2=c1 THEN 370
170 FOR c3=1 TO 7
180 IF c3=c2 OR c3=c1 THEN 360
190 FOR c4=1 TO 7
200 IF c4=c3 OR c4=c2 OR c4=c1 THEN 350
210 FOR c5=1 TO 7
220 IF c5=c4 OR c5=c3 OR c5=c2 OR c5=c1 THEN 340
230 FOR c6=1 TO 7
240 IF c6=c5 OR c6=c4 OR c6=c3 OR c6=c2 OR c6=c1 THEN 330
250 FOR c7=1 TO 7
260 IF c7=c6 OR c7=c5 OR c7=c4 OR c7=c3 OR c7=c2 OR c7=c1 THEN 320
270 o$=MID$(s$,c1,1)+MID$(s$,c2,1)+MID$(s$,c3,1)+MID$(s$,c4,1)+ MID$
(s$,c5,1)+MID$(s$,c6,1)+MID$(s$,c7,1)
280 LOCATE 18,5:PRINT x;o$
290 |BANKWRITE,r%,o$
300 IF r%<0 THEN STOP
310 x=x+1
320 NEXT c7
330 NEXT c6
340 NEXT c5
350 NEXT c4
360 NEXT c3
370 NEXT c2
380 NEXT c1
390 ultimereg=r%
400 REM ahora vamos a buscar
410 r%=0:g$=SPACE$(7)
420 PRINT:INPUT"Que subpalabra quiere buscar (puede escribir ? como simbolo
comodin) ",m$
430 m$=LEFT$(m$,7)
440 FOR x=1 TO LEN(m$)
450 IF MID$(m$,x,1)="?"THEN MID$(m$,x,1)=CHR$(0)
460 NEXT
470 |BANKFIND,r%,m$,0,ultimereg
480 IF r%<0 THEN 420
490 |BANKREAD,r%,g$
500 PRINT g$,
510 |BANKFIND,r%,m$,r%+1,ultimereg
520 GOTO 480

```